A ICPC Problems

请输入文本

```
void solve() {
    int n;
    cin >> n;
    for (int i = 0; i < n; ++i) cout << (char)('A' + i) << " ";
    cout << "\n";
}
```

Fence 1

B Chess

对于象，它可以移动到 $(\pm 2, \pm 2)$ 的位置，也就是说

- 如果 $n \le 2$ 或 $m \le 2$ ，必然一步都跳不了
- 移动不会改变它横坐标和纵坐标的奇偶性。

记 $a = \lceil \frac{n}{2} \rceil, b = \lfloor \frac{n}{2} \rfloor, c = \lceil \frac{m}{2} \rceil, d = \lfloor \frac{m}{2} \rfloor$ 我们可以把棋盘分为四个子块：

- 与 $(1, 1)$ 奇偶性相同的点，数量为 $a \times c$ 。
- 与 $(1, 2)$ 奇偶性相同的点，数量为 $a \times d$ 。
- 与 $(2, 1)$ 奇偶性相同的点，数量为 $b \times c$ 。
- 与 $(2, 2)$ 奇偶性相同的点，数量为 $b \times d$ 。

对角线移动又会保证 $(x + y) \mod 4$ 的值一定，在奇偶性相同的条件下，每块又会被分为两个连通块，最大连通块的大小为 $\lceil \frac{size}{2} \rceil$ 。

枚举四个起点取最大值就行。

```cpp
void solve() {
    ll n, m;
    cin >> n >> m;
    if (n <= 2 || m <= 2) cout << 1 << "\n";
    else {
        ll a1 = (n + 1) / 2, a2 = n / 2, b1 = (m + 1) / 2, b2 = m / 2;
        cout << max({(a1 * b1 + 1) / 2, (a1 * b2 + 1) / 2, (a2 * b1 + 1) / 2, (a2 * b1 + 1) / 2}) << "\n";
    }
}
```

C Sequence Cost

记数组和为 S ，不难猜到最终的答案为 $min(S, mx + n * mn)$ 。接下来给出证明：

如果不进行修改，代价为 S ；

如果进行修改，记操作花费为 C ，最终数组的和为 F ：

- 整个数组的 mx 被修改了，此时必然有 $C \geq mx$ ，$C + F \geq mx + n * mn$ 。
- 整个数组的 mx 没有被修改，最终的数组的 sum 至少为 $mx + (n-1) * mn$ ，此时总费用为 $C + mx + (n-1) * mn$ ：
  - 如果 $S \geq mx + n * mn$ ， $C + mx + (n-1) * mn - (mx + n * mn) = C - mn$ ，如果进行了有效的修改， C 显然大于 mn ，故 $C + mx + (n-1) * mn \geq mx + n * mn$ 。
  - 如果 $S < mx + n * mn$ ，即 $mx > S - n * mn$ ，$C + mx + (n-1) * mn - S \geq C - mn$ ，如果进行了有效的修改， C 显然大于 mn ，$C + mx + (n-1) * mn \geq mx + n * mn$ 。

综上，最小代价为 $min(S, mx + n * mn)$ 。

```cpp
void solve() {
    ll n, mn = INF, mx = 0, sum = 0;
    cin >> n;
    for (ll i = 0, x; i < n; ++i) cin >> x, sum += x, mn = min(mn, x), mx = max(mx, x);
    cout << min(mx + mn * n, sum) << "\n";
}
```

D Digital Deletion

如果当前的 mex 为 $R+1$，即前面的数字可以组成 $1 \sim R$ 的所有数字，如果引入一个新的数字 x，R 就能被扩展到 $R+x$：

- 对于 $t \in [1, x]$，还是原来的组合方式
- 对于 $t \in [x+1, R]$，可以通过 $x + [t-x]$ 的组合得到。（此处 $[x]$ 为组成 x 的一组方案）

用这种方式，我们就能把 R 拓展到 $R+x$，此时要求 $x \le R+1$。

这样从小到大模拟一遍就可以算出最大的 R 了。

- 如果 R 为 0，显然所有数字都能被删除。
- 对于其余情况：
  - 如果 $x=0$ 或 $x>R$，它产生不了任何贡献，可以直接删。
  - 对于剩余的情况，我们贪心地拿出 $\le cur+1$ 的最大的数进行更新，直到 $cur=R$，剩下的数字就是能删的数字。

```cpp
void solve() {
    int n;
    cin >> n;
    vector<ll> a(n);
    for (int i = 0; i < n; ++i) cin >> a[i];
    sort(all(a));

    ll R = 0;
    for (int i = 0; i < n; ++i) {
        if (a[i] <= R + 1) R += a[i];
        else break;
    }
    if (R == 0) {
        cout << n << "\n";
        return;
    }

    multiset<ll> ms;
    for (int i = 0; i < n; ++i) {
        if (1 <= a[i] && a[i] <= R) ms.insert(a[i]);
    }

    ll cur = 0;
    int keep = 0;
    while (cur < R) {
        auto it = ms.upper_bound(cur + 1);
        if (it == ms.begin()) {
            break;
        }
        --it;
        cur += *it;
        ms.erase(it);
        ++keep;
    }
    cout << n - keep << "\n";
```

```
}
```

Fence 4

E Block Array

好数组就是连接的几个块，所以我们贪心地进行分割，若当前位置值为 k ，则要求区间 $[i, i+k-1]$ 内所有值都等于 k ，若满足则把这段当作一个块，下一起点变为 $i+k$ 。对每个 i 记录能连续切出的块个数 $\mathrm{cnt[i]}$ ，那么以 i 为左端的好子数组个数就是 $\mathrm{cnt[i]}$ ，总体答案就是 $\sum_i cnt[i]$ 。

```cpp
void solve() {
    int n;
    cin >> n;
    vector<int> a(n + 2, 0);
    for (int i = 1; i <= n; ++i) cin >> a[i];
    vector<int> R(n + 2, 0);
    R[n + 1] = n + 1, R[n] = n;
    for (int i = n; i >= 1; --i) {
        if (i < n && a[i] == a[i + 1]) R[i] = R[i + 1];
        else R[i] = i;
    }
    vector<int> cnt(n + 3, 0);
    ll ans = 0;
    for (int i = n; i >= 1; --i) {
        int k = a[i], r = i + k - 1;
        if (k <= 0 || r > n) {
            cnt[i] = 0;
        } else {
            if (R[i] >= r) {
                int j = i + k;
                if (j <= n) cnt[i] = 1 + cnt[j];
                else cnt[i] = 1;
            } else {
                cnt[i] = 0;
            }
        }
        ans += cnt[i];
    }
    cout << ans << "\n";
}
```

F

我们需要构建一个字典序最大的序列。这意味着我们希望从左到右尽可能让每一个位置的数字最大。在每一个位置 i（当前还没有确定的位置），我们有两个选择：

1. 延续上一次操作的值：如果我们之前进行了一个操作，当前位置 i 可以被包含在那个操作的延伸区间中（或者被一个新的与之前值相同的操作覆盖，代价为 1 ）。如果之前的“活跃值” V 比我们在当前预算 k 下能找到的新的最大值还要大，那么花费 1 的代价将 V 延伸到 i 是最优的。
2. 开始一个新的区间操作：如果在当前位置 i 和预算 k 允许的范围 $[i, i+k]$ 内，能找到一个比 V 更大的值（或者没有活跃值，或者 V 不够大），我们就应该去获取这个最大值。假设最大值 M 出现在下标 p ，我们执行操作区间 $[i, p]$ 。这样 $a[i \ldots p]$ 都会变成 M ，花费是 $p-i$ 。为了给后面的操作保留尽可能多的预算，如果有多个相同的最大值 M ，我们选择最左边的一个（下标 p 最小）。

```cpp
void solve() {
    int n;
    ll k;
    cin >> n >> k;
    ST st(n);
    for (int i = 1; i <= n; i++) {
        cin >> st.in1[i];
    }
    st.init();

    vector<int> ans(n + 1);
    int i = 1;
    int las = -1;

    while (i <= n) {
        ll lim = i + k;
        int r = (lim > n) ? n : lim;

        auto [mv, pos] = st.getMax(i, r);
        pos = -pos;

        if (las > mv && k >= 1) {
            ans[i] = las;
            k--;
            i++;
        } else {
            int cost = pos - i;

            for (int j = i; j <= pos; j++) {
                ans[j] = mv;
            }

            k -= cost;
            las = mv;
            i = pos + 1;
        }
    }

    for (int j = 1; j <= n; j++) {
```

```
        cout << ans[j] << " ";
    }
    cout << "\n";
}
```

Fence 6

头文件

```cpp
//Another
#include<bits/stdc++.h>
#include<bits/extc++.h>
#define pb push_back
#define eb emplace_back
#define fi first
#define se second
#define all(a) a.begin(), a.end()
#define rall(a) a.rbegin(), a.rend()
using namespace std;

typedef long long ll;
typedef long double ld;
typedef unsigned long long ull;
typedef __int128 i128;
typedef pair<int, int> PII;
typedef pair<ll, ll> PLL;
typedef tuple<ll, ll, ll> TLLL;
typedef __gnu_pbds::tree<PLL, __gnu_pbds::null_type, less<PLL>, __gnu_pbds::rb_tree_tag, __gnu_pbds::tree_order_statistics_node_update> Tree;
// typedef __gnu_pbds::tree<ll, __gnu_pbds::null_type, less<ll>, __gnu_pbds::rb_tree_tag, __gnu_pbds::tree_order_statistics_node_update> Tree;

constexpr int inf = (ll)1e9 + 7;
constexpr ll INF = (ll)2e18 + 9;
// constexpr ll INF = (ll)4e18;
// constexpr ll MOD = 1e9 + 7;
constexpr ll MOD = 998244353;
constexpr ld PI = acos(-1.0);
constexpr ld eps = 1e-10;

mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
ull randint(ull l, ull r) {uniform_int_distribution<unsigned long long> dist(l, r); return dist(rng);}

void init() {

}

void solve() {

}

int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);

    init();

    int t = 1;
```

```
    cin >> t;
    for (int _ = 1; _ <= t; ++_) {
        solve();
    }

    return 0;
}
```

ST表（额外记录了max的下标）

```cpp
struct ST {
    const int n, k;
    vector<int> in1, in2;
    vector<vector<PII>> Max;
    vector<vector<int>> Min;
    ST(int n) : n(n), in1(n + 1), in2(n + 1), k(__lg(n)) {
        Max.resize(k + 1, vector<PII>(n + 1));
        Min.resize(k + 1, vector<int>(n + 1));
    }
    void init() {
        for (int i = 1; i <= n; i++) {
            Max[0][i] = {in1[i], -i};
            Min[0][i] = in2[i];
        }
        for (int i = 0, t = 1; i < k; i++, t <<= 1) {
            const int T = n - (t << 1) + 1;
            for (int j = 1; j <= T; j++) {
                Max[i + 1][j] = max(Max[i][j], Max[i][j + t]);
                Min[i + 1][j] = min(Min[i][j], Min[i][j + t]);
            }
        }
    }
    pair<int, int> getMax(int l, int r) {
        if (l > r) {
            swap(l, r);
        }
        int k = __lg(r - l + 1);
        return max(Max[k][l], Max[k][r - (1 << k) + 1]);
    }
    int getMin(int l, int r) {
        if (l > r) {
            swap(l, r);
        }
        int k = __lg(r - l + 1);
        return min(Min[k][l], Min[k][r - (1 << k) + 1]);
    }
};
```